

Express Validator

Datos del lado del servidor

A la hora de trabajar con datos por parte del servidor, es importante tener ciertas consideraciones para que el tratamiento de dichos datos sea el más adecuado, ya que estas consideraciones sólo son aplicables al lado del servidor.

Teniendo en cuenta que los datos que nos son proveídos al servidor en la mayoría de los casos forman parte de la información que se guardará en la base de datos, es fundamental tener los siguientes conceptos presentes:

Integridad de los Datos

La integridad de los datos se refiere a la precisión, la coherencia y la confiabilidad de la información almacenada en una aplicación. Es esencial porque los datos incorrectos o maliciosamente manipulados pueden tener consecuencias graves, como pérdida de confianza de los usuarios, toma de decisiones erróneas y daños a la reputación de la empresa. Garantizar la integridad de los datos implica validar y sanear las entradas de los usuarios para asegurarse de que cumplan con ciertos criterios y restricciones antes de ser almacenados o procesados.

Mantenimiento Simplificado

El mantenimiento de una aplicación es un aspecto crítico para su éxito a largo plazo. Un código limpio y organizado facilita la adición de nuevas características, la corrección de errores y la realización de actualizaciones. Si la validación y el saneamiento de datos se manejan de manera fragmentada en diferentes partes de la aplicación, el mantenimiento puede volverse complicado y propenso a errores. Un enfoque estructurado, como express-validator, ayuda a mantener un código más claro y coherente, lo que simplifica el proceso de mantenimiento y mejora la escalabilidad.

Middlewares

Un middleware es una función que se ejecuta durante el recorrido de una petición HTTP, antes de que dicha petición llegue al controlador o función que genera la respuesta final.

Los middlewares se utilizan para realizar tareas intermedias como:

- **Validar datos.**
- **Verificar autenticación.**
- **Comprobar permisos.**
- **Registrar peticiones.**
- **Manejar errores.**

Generalmente reciben los parámetros **req**, **res** y **next**:

```
const middleware = (req, res, next) => {  
  // Lógica del middleware  
  
  next();  
};
```

La función **next()** permite continuar con la lógica del siguiente middleware o controlador.

Express Validator utiliza justamente este mecanismo, permitiendo ejecutar validaciones como middlewares antes de que los datos lleguen al controlador.

Express-validator y sus Ventajas

express-validator es una biblioteca de validación y saneamiento de datos diseñada específicamente para aplicaciones web basadas en Express. Aquí hay algunas ventajas clave de utilizar esta biblioteca:

- **Seguridad Mejorada:** express-validator te permite validar los datos de entrada y prevenir ataques maliciosos, como inyecciones SQL y ataques de scripts entre sitios (XSS), al asegurarte de que los datos cumplan con los requisitos necesarios antes de ser procesados.
- **Integridad de los Datos:** La validación precisa ayuda a mantener la calidad y la confiabilidad de los datos almacenados en tu aplicación, evitando problemas derivados de datos incorrectos o no válidos.
- **Experiencia del Usuario:** Al proporcionar mensajes claros de error y guiar a los usuarios para corregir sus entradas, puedes mejorar la experiencia del usuario y reducir la frustración.
- **Mantenimiento Simplificado:** express-validator te permite centralizar y estandarizar la lógica de validación en tu aplicación, lo que facilita el mantenimiento continuo y la expansión futura.
- **Facilidad de Uso:** La sintaxis simple y expresiva de express-validator facilita la definición de reglas de validación y su integración en tus rutas y controladores existentes.
- **Prevención de Errores:** Al garantizar que los datos cumplen con los requisitos necesarios antes de ser procesados, puedes prevenir errores y excepciones inesperados en tu aplicación.

¿Cómo puedo usar express-validator?

Lo primero, para poder hacer uso de esta librería, es instalarla como cualquier dependencia de npm.

```
npm install express-validator
```

Uso básico con body()

Una vez instalada la librería, una de las formas más fáciles de usar **express-validator** es aplicar una de las funcionalidades que nos provee, la cual es **"body"**. Como su nombre lo indica, sirve para validar los datos que provienen de **request.body** o (**req.body**), es decir, para validar los datos de entrada del cuerpo de la petición.

Esta se puede utilizar directamente como un middleware y la misma es una función que recibe como parámetro el nombre del campo al que deseamos validar.

```
import { body } from 'express-validator';

app.post('/users', body('email'), controller);
```

Y luego a través de métodos que posee la propia función, podemos concatenar una serie de validaciones como por ejemplo que no esté vacío con el método, **.notEmpty()**:

```
app.post('/users', body('email').notEmpty(), controller);
```

Seguido de esto, también podemos darle un mensaje personalizado de error en caso de que el email esté vacío de la siguiente forma:

```
app.post(
  '/users',
  body('email').notEmpty().withMessage('Email is required'),
  controller
);
```

Validando errores con validationResult()

Para probar si esto funciona correctamente, debemos utilizar una función propia de **express-validator** para chequear si existen errores en el body o no y esta se llama **"validationResult"**, la cual recibe como parámetro la request de la petición por lo tanto se puede usar en el mismo controlador de la petición y de esa manera controlar la respuesta del servidor en caso de que el mismo contenga errores, esto lo haremos con el método de **isEmpty()**:

Crearemos una condición en la cuál retornaremos una respuesta con un status code de error en caso de que los errores no estén vacíos.

```
import { validationResult } from 'express-validator';

const controller = (req, res) => {
  const errors = validationResult(req);

  if (!errors.isEmpty()) {
    return res.status(400).json(errors);
  }

  res.status(200).send('User created successfully');
};
```

Y en caso de tener un error, tendríamos una respuesta como ésta:

```
{
  "errors": [
    {
      "type": "field",
      "msg": "Email is required",
      "path": "email",
      "location": "body"
    }
  ]
}
```

Middleware personalizado para validaciones

Dado a que la lógica que hicimos en el controlador para validar si existen errores se debería repetir en demasiados controladores de demasiadas rutas, lo mejor sería sacarlo a un middleware personalizado como por ejemplo:

```
const validate = (req, res, next) => {  
  const errors = validationResult(req);  
  
  if (!errors.isEmpty()) {  
    return res.status(400).json(errors);  
  }  
  
  next();  
};
```

Y así utilizarlo en todas las rutas en las que sea necesario sin repetir todo el código dentro del controlador, que es justo lo que queremos evitar utilizando **express-validator**.

La ruta nos quedaría algo como ésto:

```
app.post(  
  '/users',  
  body('email').notEmpty().withMessage('Email is required'),  
  validate,  
  controller  
);
```

Validaciones múltiples y concatenadas

Pero, en este caso, solo estamos validando que el campo **"email"** no esté vacío. ¿Qué pasaría si quisiéramos validar más cosas, como por ejemplo que sea un correo electrónico válido? En ese caso, la ruta nos quedaría algo como esto:

```
app.post(  
  '/users',  
  body('email')  
    .notEmpty().withMessage('Email is required')  
    .isEmail().withMessage('Email must be valid'),  
  validate,  
  controller  
);
```

En el código, se pueden concatenar validaciones, una después de la otra. ¿Qué pasa si queremos validar otros campos?

Tendríamos que agregar las validaciones una por una:

```
app.post(
  '/users',
  body('email')
    .notEmpty().withMessage('Email is required')
    .isEmail().withMessage('Email must be valid'),
  body('password')
    .notEmpty().withMessage('Password is required')
    .isLength({ min: 8 }).withMessage('Password must be at least 8
characters'),
  body('firstName')
    .notEmpty().withMessage('First name is required')
    .isLength({ min: 2 }).withMessage('First name must be at least 2
characters'),
  body('lastName')
    .notEmpty().withMessage('Last name is required')
    .isLength({ min: 2 }).withMessage('Last name must be at least 2
characters'),
  body('phone')
    .notEmpty().withMessage('Phone is required')
    .isLength({ min: 8 }).withMessage('Phone must be at least 8
characters'),
  validate,
  controller
);
```

Organizando validaciones en arrays

En el código anterior podemos ver cómo quedaría si hacemos las validaciones una por una dentro del cuerpo de la definición de dicha ruta. En este caso lo mejor sería extraer esas validaciones en otra parte y gracias a cómo funciona **express-validator** y el propio express, se puede lograr.

Esto se puede hacer de dos maneras:

Primera forma: Array de validaciones

La primera es simplemente definiendo estas validaciones dentro de un array y pasarle ese array a la ruta como si fuese un middleware de la siguiente forma:

```
const validations = [
  body('email')
    .notEmpty().withMessage('Email is required')
    .isEmail().withMessage('Email must be valid'),
  body('password')
    .notEmpty().withMessage('Password is required')
    .isLength({ min: 8 }).withMessage('Password must be at least 8 characters'),
  body('firstName')
    .notEmpty().withMessage('First name is required')
    .isLength({ min: 2 }).withMessage('First name must be at least 2 characters'),
  body('lastName')
    .notEmpty().withMessage('Last name is required')
    .isLength({ min: 2 }).withMessage('Last name must be at least 2 characters'),
  body('phone')
    .notEmpty().withMessage('Phone is required')
    .isLength({ min: 8 }).withMessage('Phone must be at least 8 characters')
];
```

Y en la ruta importamos ese array:

```
app.post(
  '/users',
  validations,
  validate,
  controller
);
```

Segunda forma: checkSchema

La segunda forma es una que nos provee **express-validator** y es con un método especial que se llama "**checkSchema**" y se utiliza de la siguiente forma:

```
import { checkSchema } from 'express-validator';

const validations2 = checkSchema({
  email: {
    notEmpty: {
      errorMessage: 'Email is required'
    },
    isEmail: {
      errorMessage: 'Email must be valid'
    }
  }
});
```

Y si hacemos todas las validaciones como en el otro método, quedaría de esta forma:

```
const validations2 = checkSchema({
  email: {
    notEmpty: {
      errorMessage: 'Email is required'
    },
    isEmail: {
      errorMessage: 'Email must be valid'
    }
  },
  password: {
    notEmpty: {
      errorMessage: 'Password is required'
    },
    isLength: {
      errorMessage: 'Password must be at least 8 characters',
      options: { min: 8 }
    }
  }
  // rest of validations
});
```


Y la ruta de esta manera:

```
app.post(
  '/users',
  validations2,
  validate,
  controller
);
```

Con este método, podemos ver que las rutas quedan mucho más limpias y ordenadas, ya que tanto el middleware de verificación de errores, el controlador y las validaciones se pueden crear en otros archivos y utilizarlos simplemente importando donde sea necesario.

Ambas formas son igualmente funcionales, pero la ventaja del primer método es que al estar utilizando las funciones que nos proporciona "**express-validator**" concatenadas una detrás de la otra, el autocompletado funciona de mejor manera con Visual Studio Code. Por ende, se recomienda emplear el primer método.

Método `matchedData()`

El método **`matchedData()`** es una utilidad muy importante de **express-validator** que nos permite obtener únicamente los datos que han pasado la validación, filtrando automáticamente cualquier campo adicional que no haya sido validado.

¿Por qué usar `matchedData()`?

Cuando los usuarios envían datos a nuestro servidor, pueden incluir campos adicionales que no esperamos o que no queremos procesar. **`matchedData()`** nos ayuda a:

- **Seguridad:** Previene que datos no validados lleguen a nuestra lógica de negocio
- **Limpieza:** Nos asegura que solo trabajemos con los datos que hemos validado explícitamente
- **Consistencia:** Garantiza que la estructura de datos sea predecible

Uso básico de matchedData()

```
import { body, validationResult, matchedData } from
'express-validator';

const controller = (req, res) => {
  const errors = validationResult(req);

  if (!errors.isEmpty()) {
    return res.status(400).json(errors);
  }

  // Obtener solo los datos que pasaron la validación
  const validatedData = matchedData(req);
  console.log(validatedData); // Solo campos validados

  // Usar validatedData en lugar de req.body
  // createUser(validatedData);

  res.status(200).json({ message: 'User created', data:
validatedData });
};
```

Ejemplo práctico

Si tenemos estas validaciones:

```
const userValidations = [
  body('email').isEmail().withMessage('Email must be valid'),
  body('password').isLength({ min: 8 }).withMessage('Password must
be at least 8 characters'),
  body('firstName').notEmpty().withMessage('First name is required')
];
```

Y el usuario envía este JSON:

```
{
  "email": "user@example.com",
  "password": "mypassword123",
  "firstName": "John",
  "maliciousField": "hack attempt",
  "anotherField": "unwanted data"
}
```

Con **matchedData(req)** obtendremos solo:

```
{
  "email": "user@example.com",
  "password": "mypassword123",
  "firstName": "John"
}
```

Los campos **maliciousField** y **anotherField** serán automáticamente filtrados.

Método .custom()

El método **.custom()** nos permite crear validaciones personalizadas cuando las validaciones predefinidas de **express-validator** no son suficientes para nuestras necesidades específicas.

Sintaxis básica

```
body('fieldName').custom((value, { req }) => {
  // Lógica de validación personalizada
  // Retorna true si es válido
  // Lanza una excepción o retorna Promise.reject() si no es válido
})
```

Ejemplos de validaciones custom

1. Validar que un email no esté ya registrado:

```
body('email')
  .isEmail()
  .withMessage('Email must be valid')
  .custom(async (email) => {
    const existingUser = await User.findOne({ email });
    if (existingUser) {
      throw new Error('Email already in use');
    }
    return true;
  })
```

Validaciones custom asíncronas

Para validaciones que requieren consultas a base de datos o APIs externas, puedes usar `async/await`:

```
body('username')
  .isLength({ min: 3 })
  .withMessage('Username must be at least 3 characters')
  .custom(async (username) => {
    try {
      const user = await User.findOne({ username });
      if (user) {
        return Promise.reject('Username already taken');
      }
    } catch (error) {
      return Promise.reject('Error checking username availability');
    }
  })
```

Manejo avanzado de errores

Diferentes formas de obtener errores

Express-validator nos proporciona varios métodos para manejar y formatear los errores:

```
import { validationResult } from 'express-validator';

const controller = (req, res) => {
  const result = validationResult(req);

  // 1. Errores como array
  console.log(result.array());

  // 2. Errores mapeados por campo
  console.log(result.mapped());

  // 3. Errores formateados personalizados
  const custom = result.formatWith((err) => {
    return `${err.param}: ${err.msg}`;
  });
  console.log(custom.array());

  // 4. También podrías hacer result.throw()
  // res.status(400).json({ errors: result.array() });
};
```

Esto produciría una respuesta como:

```
{  
  "errors": {  
    "email": "Email is required",  
    "password": "Password must be at least 8 characters"  
  }  
}
```